Analyzing Job Resource Usage

Here we provide an overview of how to analyze the resources your jobs use, utilizing the commands `jobstats`, `htop`, `nvidia-smi`, and `nvtop`. Using these tools is important for both optimizing your code and only requesting the amount of resources your jobs need, leaving more resources open for others to use. A good place to start before using the commands we outline here is with the documentation on Requesting Resources.

Jobstats

Jobstats is a tool developed by Princeton Research Computing that allows users to easily check CPU, GPU, and memory efficiency of their jobs. On Engaging, we currently have the `jobstats` command installed on login nodes, with job data being collected on most of our compute nodes. This means that, if you have a job running, you can run `jobstats <JOB ID>` from a login node and get basic efficiency information about your job.

One key difference between `jobstats` and other tools like `nvtop` is that `jobstats` shows utilization in aggregate rather than real-time. For example:

```
[secorey@login007 ~]$ jobstats 9358523

================================================================================
                             Slurm Job Statistics
================================================================================
         Job ID: 9358523
   User/Account: secorey/mit_general
       Job Name: submit_mnist.sh
          State: RUNNING
          Nodes: 1
      CPU Cores: 1
     CPU Memory: 4GB (4GB per CPU-core)
  QOS/Partition: normal/mit_normal
        Cluster: eofe7
     Start Time: Wed Feb 18, 2026 at 11:29 AM
       Run Time: 00:02:30 (in progress)
     Time Limit: 12:00:00

                             Overall Utilization
================================================================================
  CPU utilization  [|||||||||||||||||||||||                              48%]
  CPU memory usage [|||||||||                                            19%]
```

```
                              Detailed Utilization
================================================================================
  CPU utilization per node (CPU time used/run time)
      node3310.inband: 00:02:24/00:05:01 (efficiency=47.8%)

  CPU memory usage per node - used/allocated
   node3310.inband: 779.2MB/4GB (389.6MB/2GB per core of 2)

                                      Notes
================================================================================
  * The overall CPU utilization of this job is 48%. This value is low compared
    to the target range of 90% and above. Please investigate the reason for
    the low efficiency. For more info:
      https://orcd-docs.mit.edu/running-jobs/application-analysis/

  * Have a nice day!
```

CPU utilization is calculated by dividing the total active CPU time by the total CPU wall time for the job, while memory usage is shown as the peak memory utilization over the course of the job.

> **Note**
>
> Sometimes Jobstats may report CPU utilization that does not go higher than 50% even if the CPU cores have been active during the majority of your job. This is due to an underlying bug where the number of CPU cores is reported as double the allocated amount.

We can also look at GPU metrics:

```
[secorey@login006 ~]$ jobstats 9580918

================================================================================
                              Slurm Job Statistics
================================================================================
         Job ID: 9580918
   User/Account: secorey/mit_general
       Job Name: sys/dashboard/sys/jupyter
          State: RUNNING
          Nodes: 1
      CPU Cores: 4
     CPU Memory: 32GB (8GB per CPU-core)
           GPUs: 1
  QOS/Partition: normal/mit_normal_gpu
        Cluster: eofe7
     Start Time: Mon Feb 23, 2026 at 9:53 AM
       Run Time: 00:05:15 (in progress)
     Time Limit: 02:00:00

                              Overall Utilization
```

```
================================================================================
  CPU utilization  [||||||                                              13%]
  CPU memory usage [|||                                                  6%]
  GPU utilization  [||||||||||||||||||||||||||||                        56%]
  GPU memory usage [||||||||||||||||||||||||||||||||||||||||||||||||||||96%]

                            Detailed Utilization
================================================================================
  CPU utilization per node (CPU time used/run time)
      node2804.inband: 00:05:20/00:42:01 (efficiency=12.7%)

  CPU memory usage per node - used/allocated
   node2804.inband: 1.9GB/32GB (249.0MB/4GB per core of 8)

  GPU utilization per node
      node2804.inband (GPU 1): 55.7%

  GPU memory usage per node - maximum used/total
      node2804.inband (GPU 1): 43.0GB/45GB (95.5%)

                                   Notes
================================================================================
  * Have a nice day!
```

The above output shows solid GPU utilization but lower CPU utilization, so we might consider lowering our CPU request for future runs.

htop

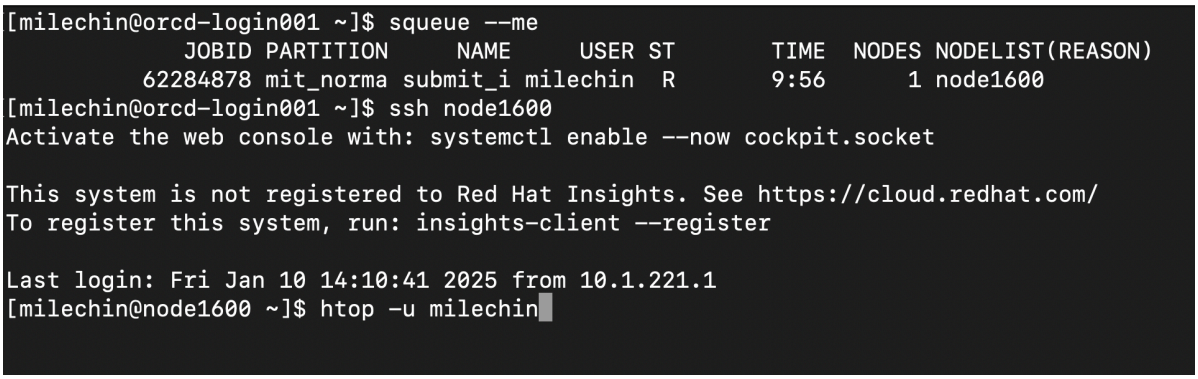
You can get a lot of information about your running jobs through the `htop` command. It sort of gives you a way to watch the compute node your job is running on work on your job. It will show you your current instantaneous CPU and Memory utilization, how many cores are being used, how many threads are running, and a list of your processes running on the node, among other things. If you are familiar with the top Linux command, it is similar but shows more information.

How to Run htop

The key thing to know is you need to run the `htop` command on the node where your job is running. First, get the name of the node your job is running on. You can do this by running the `squeue --me` command. The last column has the node name. If you are running in an interactive job, the node name will also be at your command prompt.

The next step is to SSH to the node where your job is running. Run the command `ssh NODENAME`, where NODENAME is the name of the node you just found. If you are running an interactive job, open a new terminal tab or window and log into Engaging. Note that you can only SSH to nodes

where you have a job running, and you should exit the node once you are done monitoring your job with the exit command.



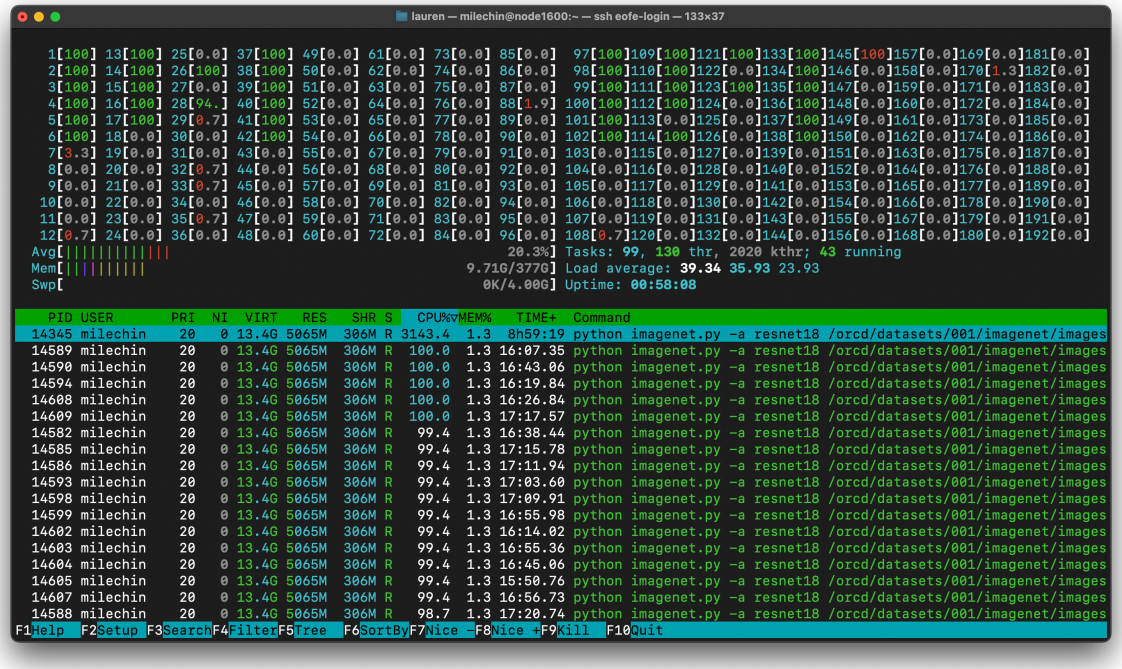
In the above image, you can see we have run `ssh node1600`, and the command prompt now indicates we are on that node. From here, we can run:

```
htop -u USERNAME
```

where USERNAME is your username. Specifying your username filters out processes that don't belong to you and makes it easier to interpret the output.

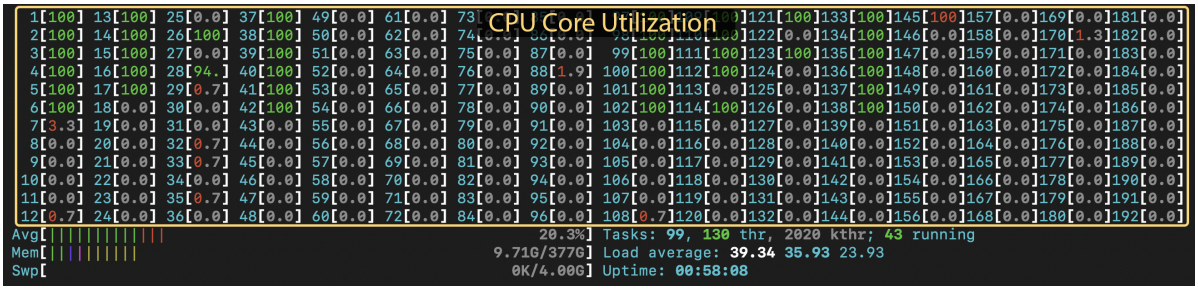
Using htop

The following is a screenshot of the htop program.



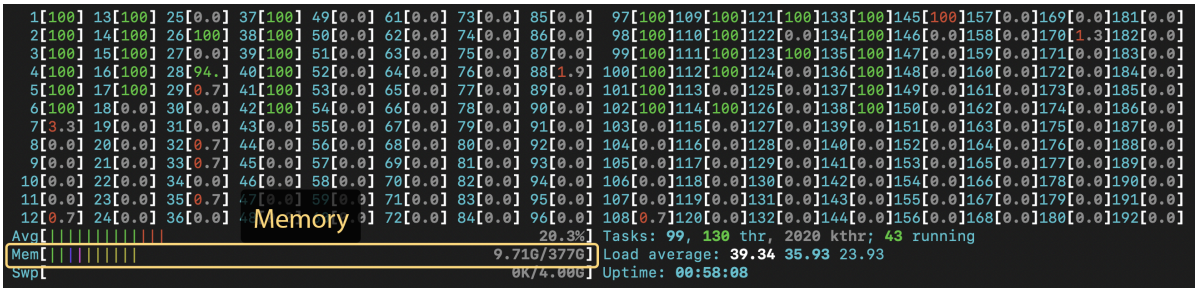
The first thing you'll notice when you run htop yourself is that it is constantly moving to give you instantaneous statistics. There are two main sections in the htop output. The bottom half shows the processes running on the node, and the top half shows the activity on the node. Note that by specifying your username, you will only see your processes, but the activity in the top half may reflect processes running on the node that aren't yours, so if you want an accurate representation of your job, you may want to request a full node.

If you are on a node with a lot of cores, it may be difficult to see the full output. You can set htop to use a more compact visualization by creating a config file. First, create the directory ~/.config/htop if it doesn't already exist. Download this file and put it in the ~/.config/htop directory. The file should be named htoprc.

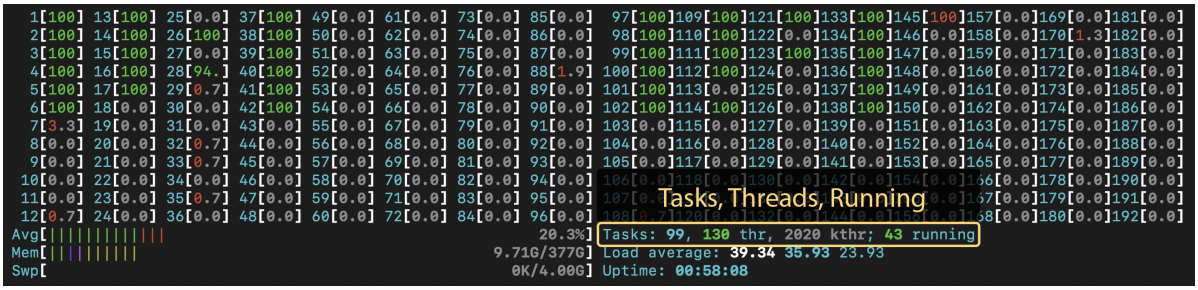


In the top half, you'll see numbered bars with percentages. These represent the cores on the node. You'll see twice the number of bars as cores. This is due to hyperthreading: each physical core has two "virtual" cores. This display gives you a rough idea of how hard the CPUs on the node are

working. If a CPU is at 100%, it is fully utilized. If there isn't much activity, not much is going on on the node. If you are expecting multiple processes or threads to be working, and you only see one bar fill up, you have an indication that you might need to change a setting. Alternatively, maybe you haven't explicitly asked your program to use multiple cores and you expect it to use only one, but you see multiple CPUs at 100%, even when you are the only person on the node. In that case, you may need to tell the package you are using how many threads or CPUs to use.



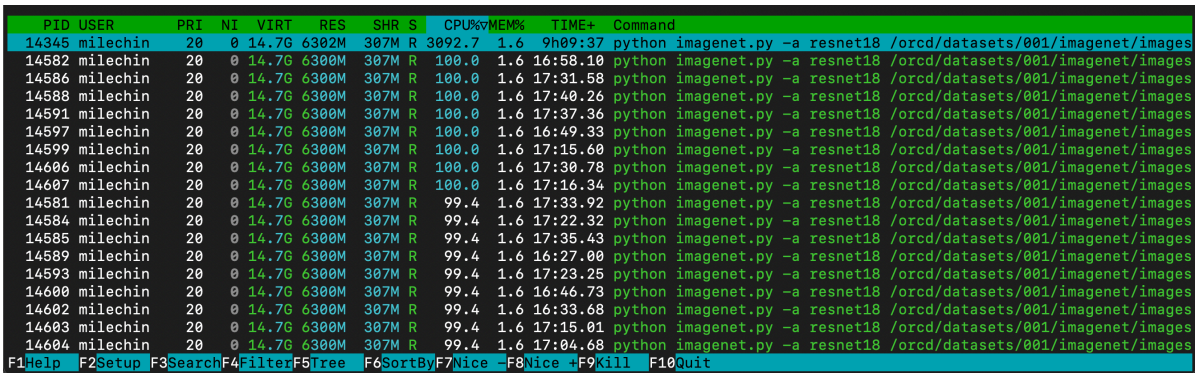
In the top half, you'll also see a bar labeled "Mem" under the CPU bars. This is the instantaneous memory usage on the node. Note that just because you see a certain number here while you are watching doesn't mean this is the maximum memory your job will use. To get that information, you'll want to use the sacct command.



Next to the "Mem" bar, you'll see some text labeled "Tasks". This lists running processes (tasks), threads, and running processes or threads. You generally don't want more running processes or threads than the number of cores on the node. Again, this number may reflect another user's threads or processes running on the node. In the screenshot above, it shows that there are 43 running threads. I requested 16 cores for the job, and with hyperthreading, the application created 32 threads. There was another user running on the node who was running their own job that added a few more threads.



Below the "Tasks" is a line that shows "Load average". It shows three numbers: the load average for the past 1, 5, and 15 minutes. The load is roughly the number of running processes, the load average over the last X minutes. This number is somewhat related to CPU utilization, as each CPU is realistically capable of running a single process at a time. It therefore gives you roughly a single-number condensed view of the CPU utilization image at the top. If this number is high in relation to the number of cores on the node, then you know those cores are working hard. If, for some reason, this number is over the number of cores, you are likely overworking the node and should scale back on the number of threads or processes you have running on that node. Engaging is configured to prevent this, but it is a good thing to keep an eye on. You may find that with fewer threads or processes, your application will actually run faster in this case, as they are no longer competing for resources.



Now let's take a quick look at the bottom half. As mentioned above, this shows all of your processes and threads running on the node. You can see the PID (process ID), username (you'll only see your own processes), state, CPU utilization %, memory usage %, and the command that initiated the process, among other things. Clicking on the different headers re-sorts the list by that column, here I have it sorted by CPU%. In this case, I started one Python process, and that Python application created many threads. The main process happens to be at the top in this case, since I have it sorted by CPU%. When you have a multithreaded application, the CPU usage for the individual threads is included in the number for the main process. Each thread is currently using about 100% of a CPU (100% is full usage of a single CPU), so the total for the entire application is 3092%, meaning my application is using the equivalent of 30-31 full cores across all its threads. So even though I have 43 total running threads and/or processes, they aren't all using the full power of

a CPU. All of the numbers on this bottom chart are instantaneous, rather than an average, but you can see the CPU% for the main task matches up somewhat with the load average above, when taking into account another user running on the node.

nvidia-smi

The nvidia-smi command is useful for evaluating how your GPU-accelerated applications are making use of the GPU(s) that they are using. It will give you information about GPU utilization, GPU Memory utilization, and processes using the GPU.

How to Run nvidia-smi

Like htop in the previous unit, you must be on the same node as your job to run nvidia-smi to monitor it. Further, because it is a GPU utility, if you try to run nvidia-smi on a node without a GPU, you will get a "command not found" error. And, similar to htop, you will only see statistics for the GPU(s) you have allocated to your job, so if you are on a node with GPUs but didn't request any, you won't see any listed. For more information about how to request GPUs for your job, see the pages on Requesting Resources and Scheduler Overview.

First, get the name of the node your job is running on. You can do this by running the `squeue --me` command. The last column has the node name. In the image below, you can see the node name, node2804, in the last line of the squeue output. The next step is to SSH to the node where your job is running. Run the command `ssh NODENAME`, where NODENAME is the name of the node you just found. If you are running an interactive job, open a new terminal tab or window and log into Engaging and ssh to the node from there. Note that you can only ssh to nodes where you have a job running, and you should exit the node once you are done monitoring your job with the exit command.

```
[milechin@orcd-login001 ~]$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON
          62280991 mit_norma submit_m milechin  R       0:04      1 node2804
[milechin@orcd-login001 ~]$ ssh node2804
Activate the web console with: systemctl enable --now cockpit.socket

This system is not registered to Red Hat Insights. See https://cloud.redhat.com/
To register this system, run: insights-client --register

Last login: Tue Jan  7 09:20:41 2025 from 10.1.221.1
[milechin@node2804 ~]$ nvidia-smi
```

In the above image, you can see we have run `ssh node2804`, and the command prompt now indicates we are on that node. From here, we can run the nvidia-smi command as shown.

Using nvidia-smi

When you run `nvidia-smi`, you will get a single snapshot. If you would like this to continuously refresh, which is a bit more useful in practice, you can use the -l flag, which prints a refresh to the screen at a regular interval. By default, this interval is 5 seconds, or you can specify an interval yourself. For example, if you want it to refresh every 10 seconds, you would run `nvidia-smi -l 10`. Pressing Control+C on your keyboard will stop the continuous refresh.

```
[milechin@node2804 ~]$ nvidia-smi
Fri Jan 10 11:45:58 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.14              Driver Version: 550.54.14      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA L40S                    On  |   00000000:4A:00.0 Off |                    0 |
| N/A   38C    P0            184W /  350W |    8936MiB /  46068MiB |     99%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A   1511040      C   python                                     8926MiB |
+-----------------------------------------------------------------------------------------+
```

Similar to htop, the top half of nvidia-smi shows you information about the GPU(s), and the bottom half lists the process(es) using them.

```
[milechin@node2804 ~]$ nvidia-smi
Fri Jan 10 11:45:58 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.14              Driver Version: 550.54.14      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA L40S                    On  |   00000000:4A:00.0 Off |                    0 |
| N/A   38C    P0            184W /  350W |    8936MiB /  46068MiB |     99%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A   1511040      C   python                                     8926MiB |
+-----------------------------------------------------------------------------------------+
```

The first box, circled above, tells you about the GPU, its state, and some of its physical properties. You can see the GPU name (L40S), its current temperature (38 °C), and current power usage and

power cap (using 184W, capped at 350W). This is more informational than anything, but knowing how much power your GPU is using might be something you want to think about, especially if you are using them often.

```
[milechin@node2804 ~]$ nvidia-smi
Fri Jan 10 11:45:58 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.14              Driver Version: 550.54.14      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |                  MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA L40S                    On  |   00000000:4A:00.0 Off |                    0 |
| N/A   38C    P0            184W /  350W |    8936MiB /  46068MiB |     99%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A   1511040      C   python                                      8926MiB |
+-----------------------------------------------------------------------------------------+
```

The most useful part of the second box is the memory usage. It shows both the memory you have used (roughly 8.9GB here) and the total memory on the GPU, 46GB.

```
[milechin@node2804 ~]$ nvidia-smi
Fri Jan 10 11:45:58 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.14              Driver Version: 550.54.14      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |                  MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA L40S                    On  |   00000000:4A:00.0 Off |                    0 |
| N/A   38C    P0            184W /  350W |    8936MiB /  46068MiB |     99%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A   1511040      C   python                                      8926MiB |
+-----------------------------------------------------------------------------------------+
```

The final box at the top shows the GPU utilization. You can think of this as how much you are making use of the GPU. You want to aim for 100% utilization, which is when you are getting the most out of it. If your utilization is low and if the memory allows, you may want to give the GPU more data to work on. You could also train multiple models on the same GPU. If your GPU utilization is low, it may also be a sign that your application doesn't have enough work for the GPU

to do, and you should check the speedup you are getting with the GPU vs CPUs. If it is modest, say 2x or 4x, then it isn't worth using GPUs, and you can likely get more performance by scaling out to more CPUs.

```
[milechin@node2804 ~]$ nvidia-smi
Fri Jan 10 11:45:58 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.14              Driver Version: 550.54.14      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA L40S                    On  |   00000000:4A:00.0 Off |                    0 |
| N/A   38C    P0            184W /  350W |    8936MiB /  46068MiB |     99%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A   1511040      C   python                                     8926MiB |
+-----------------------------------------------------------------------------------------+
```

The final table at the bottom shows the process(es) running on the GPU(s) you have allocated. You'll have the GPU number in the first column. If you have asked for one GPU, this will always be a "0". Type refers to whether the process is a compute (C) or graphics (G) task. You'll have the Process Name, which will be "python" if you are running a Python script, as seen here. The last column shows the GPU Memory Usage.

nvtop

`nvtop` is like `htop`, except for monitoring GPU usage. Like `nvidia-smi`, `nvtop` enables you to see the real-time statistics of your job on the GPU, including memory, compute, and power consumption. It additionally shows a sliding history of GPU compute and memory usage as well as a snapshot of CPU usage. `nvtop` in general shows more information than `nvidia-smi` and has a different look. It is up to the user's preference to pick one or use both.

How to Run nvtop

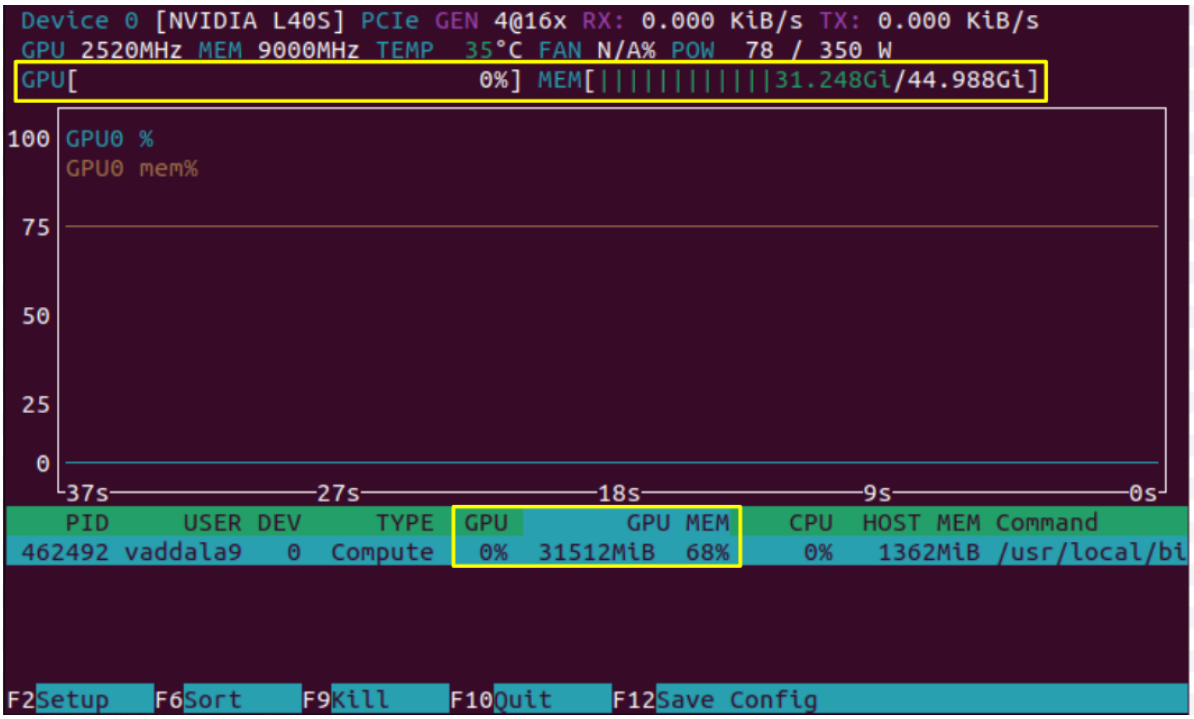
The principles of usage are very similar to those of htop, and we will outline them here. First, ssh into the node on which you are running a job using a GPU (`ssh NODENAME`) and run `nvtop`. You can refer to the htop section above for more details on finding the nodename as well as the SSH process.

Using nvtop

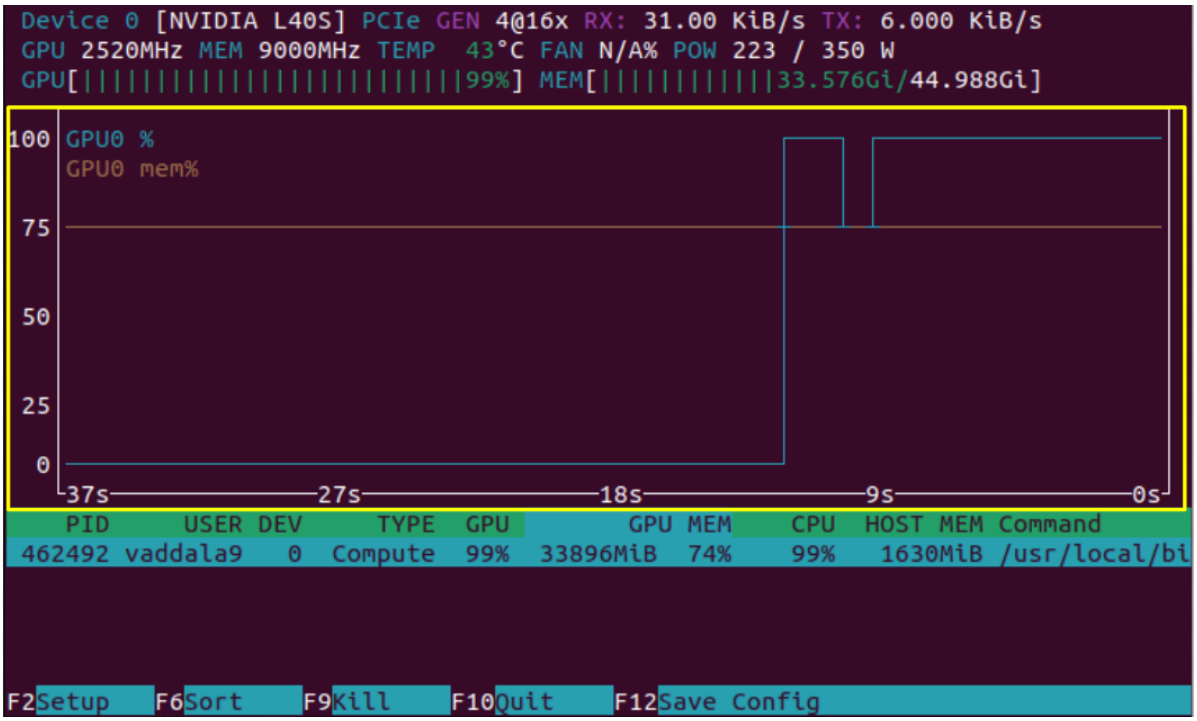
After running nvtop, you will see your usage updating in real time. Below we show an example of using this to monitor the GPU usage of RAG. If you don't yet have a GPU application and would like to get a sense of how to use nvtop, RAG could be a good example to start with.



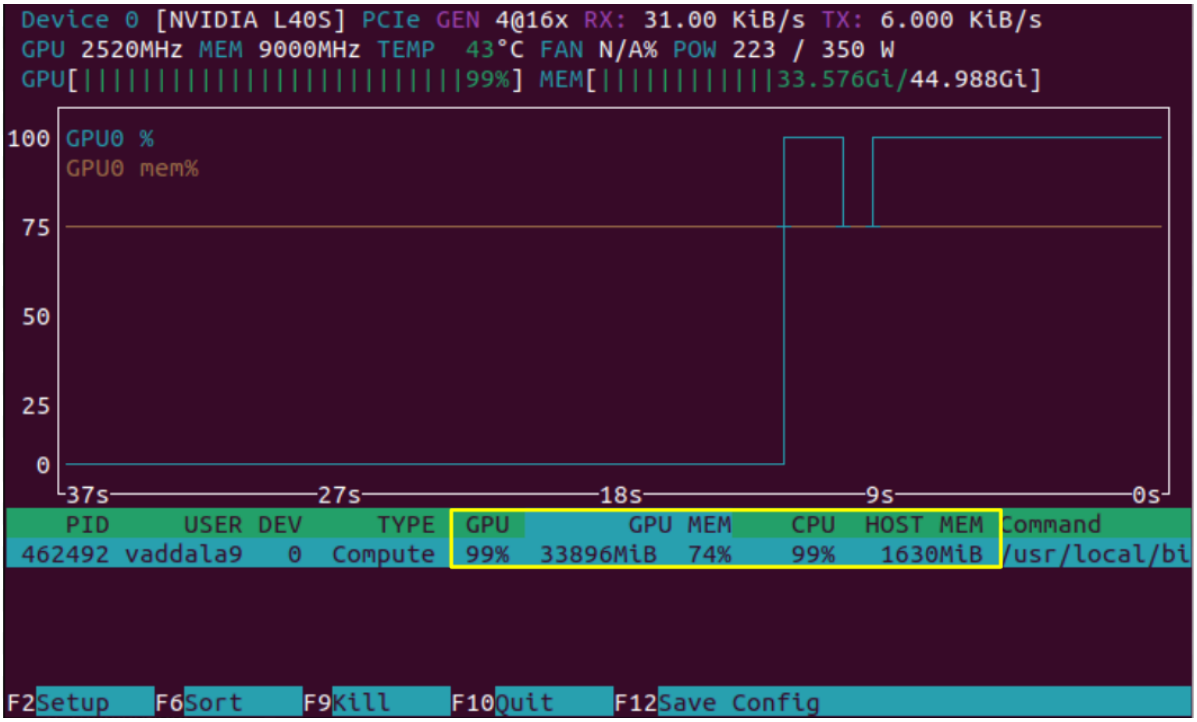
Above, we see a display that comes up after running `nvtop`. Highlighted in yellow are some static device properties. In the top left, "Device 0" is always listed if we are only using one GPU. Next to it is "L40S", the model of GPU that we are running on, and to the right of that is the interconnect interface. In the bottom yellow box, the clock speed and memory speed of the L40S are listed.



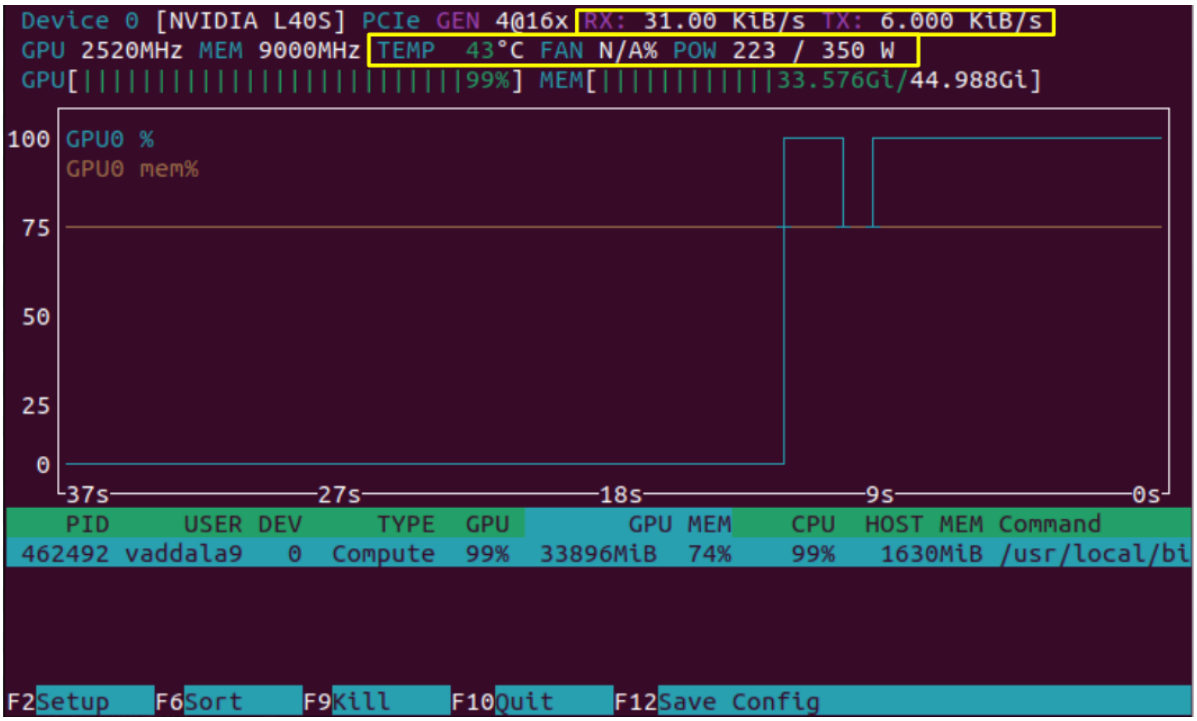
In the image above, we have only loaded the model and have not yet asked a prompt, which kicks off computation. As we expect, we see that the model takes considerable memory (31GB out of the possible 45GB), but the computing power of the GPU is not being used. These can be seen in two places on the screen, highlighted in yellow. Note, however, that some applications, like JAX, artificially fill the memory of a GPU even if that memory is not needed. This can lead to an inaccurate picture of GPU utilization, and it would be good to check whether your application could be doing this.



Now, we see the results of nvtop after we have queried the RAG model, and therefore started computation on the GPU. Highlighted is the sliding history window of GPU compute and memory usage, and as expected, we see a rise in both values.



In addition to the GPU's compute running at 99%, the CPU is also running at 99%. nvtop provides a snapshot of both GPU and CPU usage as outlined above.



On the top right, we see that "RX", data reception rate, and "TX", data transmission rate, have increased after querying the RAG model. These metrics are for data transfer between the CPU and GPU. We can also notice that the power and temperature of the system have shot up from our first look at the nvtop output. The metrics outlined in this snapshot are ones the user has less direct control over, but they can be useful for getting a better idea of how the GPU is operating.

After taking a look at all the metrics above, you could evaluate whether you want to request more or less memory in the future and whether the job can be modified to use the GPU's compute more efficiently.

March 17, 2026